

Privacy-Preserving Network Anomaly Detection

Detecting abnormal network behavior from traffic metadata
without inspecting packet payload contents

Dataset: UNSW-NB15 official split - 175,341 training rows, 82,332 test rows

Feature sets: FULL_METADATA (37) and RESTRICTED_METADATA (17)

Models: majority baseline, Logistic Regression, Random Forest

Research questions, methodology, results, limitations

Generated from this repository's configuration and results files.

Report generated: 2026-09-25

1. What this project is

This project studies whether a network detector can find abnormal behavior using only traffic metadata - flow statistics, packet headers, timing and connection state - without ever reading packet payloads. It is built as a reproducible research repository: one configuration file defines the data, the feature policy and the models; one command runs the experiment; every number in this report was read from the files that command writes.

The practical motivation is that encryption is now the default on the internet. TLS 1.3, QUIC and encrypted DNS make deep packet inspection either impossible or unacceptable, while defenders still have to find scanning, denial-of-service, exploitation and command-and-control traffic. Flow exporters such as NetFlow and IPFIX keep producing metadata regardless of what is inside the packets. This repository measures how far that metadata alone gets you on a standard public dataset.

Status of the run described in this report

Research run. The official UNSW-NB15 split was used as released: 175,341 training rows and 82,332 test rows, verified against the sha256 checksums recorded in the configuration. The two files were never merged, shuffled or re-partitioned, and nothing was tuned against the test split.

2. Research questions

Primary question: "Can network traffic metadata provide enough information to detect abnormal network behavior without inspecting packet payload contents?"

Secondary question: "How much detection performance changes when the detector is restricted to a smaller subset of traffic metadata?"

Hypothesis: metadata carries real signal for anomaly detection, and a small core subset of it keeps most of that signal while sacrificing some performance. The two feature sets in the configuration - FULL_METADATA (37 fields) and RESTRICTED_METADATA (17 fields) - are the instrument for answering the second question.

Metadata-only monitoring is not a formal privacy guarantee. The project studies payload-free detection, not complete traffic privacy.

Threat model

- Defender: observes traffic metadata only (headers, flow records, timing). Never decrypts and never parses payload.
- Attacker: produces malicious traffic and may encrypt, obfuscate or imitate legitimate flows.
- Assumption under test: an attacker cannot match the statistical profile of benign traffic across every metadata dimension at once. The experiment measures how well that holds on UNSW-NB15.

3. Dataset

UNSW-NB15 (Australian Centre for Cyber Security, 2015) mixes real normal traffic with synthetic attack traffic - DoS, exploits, fuzzers, reconnaissance, shellcode, worms, backdoors, generic - captured in a network laboratory. The repository uses its official train/test files, each with 45 columns: a row id, 42 traffic fields, the attack category and the binary label (0 = normal, 1 = anomalous) that this project uses as its target.

Split	Rows	Normal	Anomalous
Training	175,341	56,000	119,341
Testing	82,332	37,000	45,332

Checksums recorded in configs/config.yaml matched both files, so the rows above are the official release rather than a reformatted copy.

Development sample vs research dataset

- Development sample: small synthetic rows written by scripts/create_sample_data.py so tests and CI run in seconds without network access. The pipeline detects it and labels the run a smoke test on screen and in results/metrics/dataset_info.json.
- Research dataset: the official UNSW-NB15 training and testing files, recognized by row count and sha256. Only this input can produce benchmark results, and only this input should be cited.
- The sample generator refuses to overwrite the research dataset, and neither dataset is ever committed.

Obtaining the official files

python -m privacy_preserving_nad.data.download prints the current status and instructions; adding --fetch downloads the two files and verifies them against the recorded checksums. The files can also be downloaded manually from the official UNSW page (<https://research.unsw.edu.au/projects/unsw-nb15-dataset>) and placed in data/raw/. Nothing downloads automatically during tests or ordinary pipeline runs.

4. Feature policy

The policy is written down before any model is trained: traffic-level fields only, no packet contents, no raw identity. It lives in configs/config.yaml and is enforced by a guard in feature selection that aborts the run if a forbidden name appears in a feature set. docs/feature_policy.md documents every field and its justification.

Allowed categories: flow duration and timing (dur, sinpkt, dinpkt, sjit, djit, tcprtt, synack, ackdat); packet and byte counts (spkts, dpkts, sbytes, dbytes); rates and throughput (rate, sload, dload); loss counters (sloss, dloss); packet-size statistics (smean, dmean); TCP state and timing (sttl, dttl, swin, dwin, stcpb, dtcpb); connection-count aggregates computed by the dataset's exporters (ct_* and is_sm_ips_ports); and the header-level labels proto, service and state.

Excluded fields

Field	Why it is excluded
srcip	Identity or label field - not traffic behavior
dstip	Identity or label field - not traffic behavior
sport	Identity or label field - not traffic behavior
dsport	Identity or label field - not traffic behavior
srcpt	Identity or label field - not traffic behavior
dstpt	Identity or label field - not traffic behavior
Ltime	Identity or label field - not traffic behavior
Stime	Identity or label field - not traffic behavior
attack_cat	Identity or label field - not traffic behavior
trans_depth	Application-layer transaction depth. Requires HTTP parsing - excluded as payload-derived.
response_body_len	HTTP response body length. Application header inside TLS - excluded as payload-derived.
is_ftp_login	FTP login flag. Requires FTP command parsing - excluded as payload-derived.
ct_ftp_cmd	FTP command counter. Requires FTP command parsing - excluded as payload-derived.
ct_flw_http_mthd	HTTP method counter. Requires HTTP request parsing - excluded as payload-derived.

Two inclusions deserve honest wording. The ct_*_dport/sport and is_sm_ips_ports aggregates are counts keyed on addresses and ports - the raw addresses and ports themselves never become inputs. The TCP timing fields assume a visible handshake, which encrypted transports still expose. The five application-layer fields were removed from the feature sets precisely because they only exist when HTTP/FTP content was parsed; keeping them would have made the

payload-free claim false.

5. Pipeline

- 1. Data detection: identify the dataset in data/raw/ (research vs development sample) and record the outcome in results/metrics/.
- 2. Validation: schema, target values, class counts, missing values, duplicate rows and policy compliance - reported before training.
- 3. Cleaning: no rows are dropped; string fields are trimmed; any missing value would be imputed inside the sklearn pipeline.
- 4. Preprocessing: a reusable ColumnTransformer fitted on the training split only - median imputation plus standardization for numeric columns, one-hot encoding (handle_unknown='ignore') for proto, service and state - then applied to the test split.
- 5. Training: three baselines per feature set with a fixed seed.
- 6. Evaluation: metrics and confusion matrices written to results/metrics/, figures to results/figures/.
- 7. This report is generated from those same files.

Experiment design

Item	Setting
Split	official UNSW-NB15 train/test, used as released
Target	label: 0 = normal, 1 = anomalous
Random seed	None
Numeric preprocessing	median imputation + standardization (train-fitted)
Categorical preprocessing	one-hot, handle_unknown='ignore' (train-fitted)
Logistic Regression	C=1.0, max_iter=1000, solver=lbfgs, class_weight=balanced
Random Forest	n_estimators=200, max_depth=15, min_samples_split=5, min_samples_leaf=2, class_weight=balanced
Decision threshold	0.5 (default; nothing tuned on the test set)
Metrics	precision, recall, f1_score, false_positive_rate, false_negative_rate, confusion matrix

No model complexity was added on purpose. These are baselines: the point of the experiment is the information content of the metadata, not architecture search. class_weight='balanced' is used because the classes are uneven in both splits (the anomaly class is the majority), which keeps a naive all-anomalous classifier from looking competitive.

6. Results

Feature set	Model	Precision	Recall	F1	FPR	FNR
FULL_METADATA	Majority baseline	0.5506	1.0000	0.7102	1.0000	0.0000
FULL_METADATA	Logistic Regression	0.8003	0.9399	0.8645	0.2873	0.0601
FULL_METADATA	Random Forest	0.8771	0.9692	0.9208	0.1664	0.0308
RESTRICTED_METADATA	Majority baseline	0.5506	1.0000	0.7102	1.0000	0.0000
RESTRICTED_METADATA	Logistic Regression	0.7319	0.9131	0.8125	0.4097	0.0869
RESTRICTED_METADATA	Random Forest	0.8663	0.9536	0.9078	0.1804	0.0464

Class distribution

The test split holds 82,332 flows: 37,000 normal (44.9%) and 45,332 anomalous (55.1%). The anomaly class is the majority, so the majority baseline reaches F1 0.7102 by predicting 'anomalous' for everything while flagging every

normal flow as malicious (FPR 1.0000). That is the floor the real models had to clear, and both cleared it.

Best result of this run

Random Forest with FULL_METADATA: F1 0.9208 (precision 0.8771, recall 0.9692). In counts it caught 43,934 of 45,332 anomalous flows and missed 1,398 (3.1% false negatives), while flagging 6,156 of 37,000 normal flows as attacks (16.6% false positives). The false-positive column is what an operator pays for that recall.

This is a ranking inside one experiment on one dataset. It is not a claim that this model is generally superior, and no claim of privacy, production readiness or adversarial robustness follows from it.

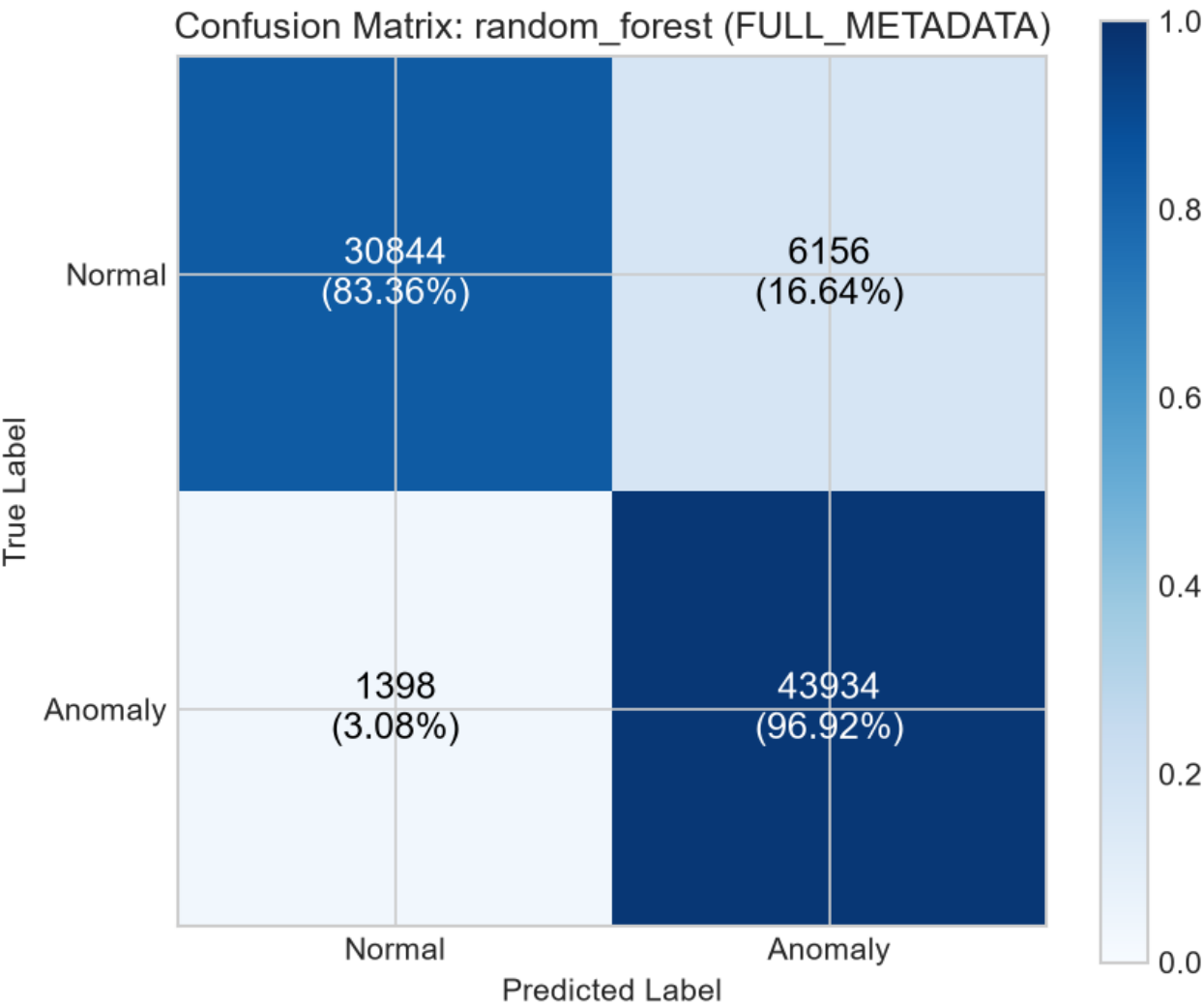
Effect of restricting the feature set

With fields cut from 37 to 17: Random Forest moved from F1 0.9208 to 0.9078 (-0.0130) and its false positive rate from 0.1664 to 0.1804; Logistic Regression moved from F1 0.8645 to 0.8125 (-0.0520) with its false positive rate from 0.2873 to 0.4097. The tree model gave up little when the feature set was cut while the linear model lost more ground: the removed fields mattered more to the model that cannot reconstruct their interactions itself. Both restricted results still sit well above the baseline, so the small feature core does carry most of the detectable signal - that is the secondary research question answered for this run.

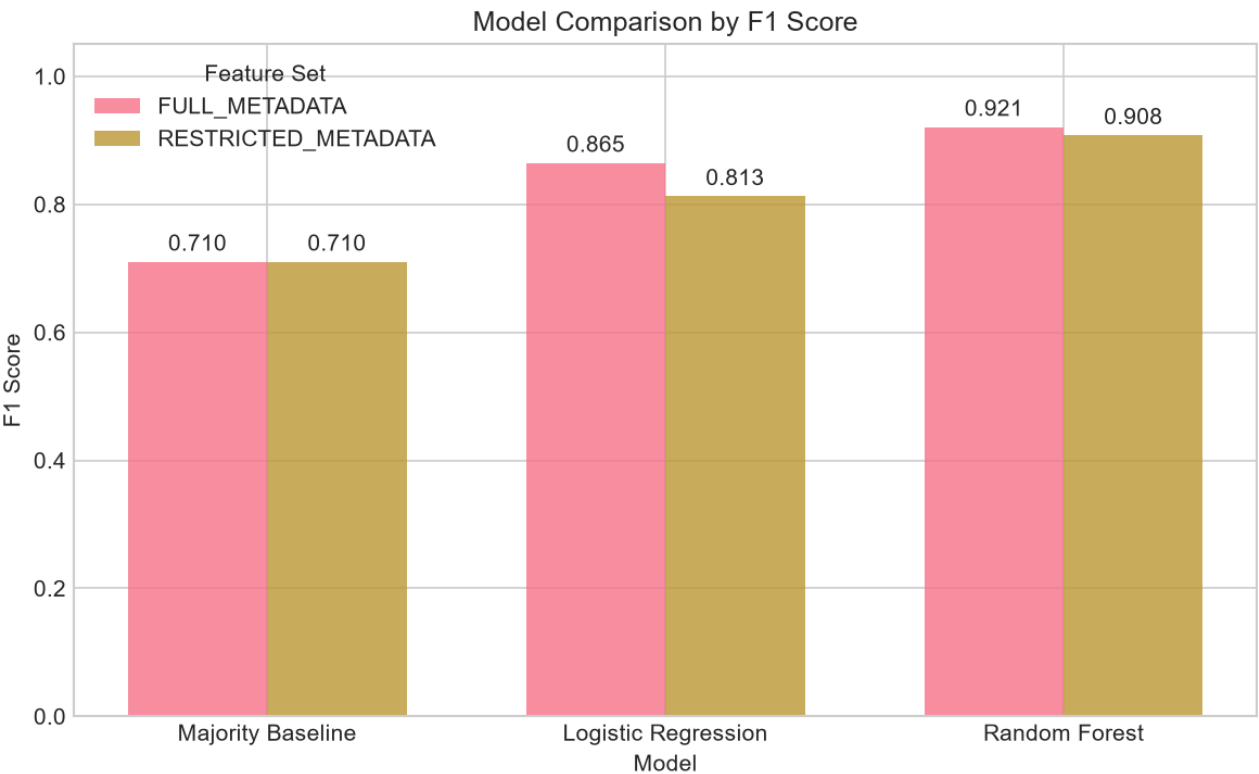
Precision, recall and the operational trade-off

Recall on the full set was 0.9399 (logistic) and 0.9692 (forest): most anomalous flows are detected, and most normal flows are not flagged - but 0.2873 and 0.1664 of normal flows respectively were. Which error is worse depends on the deployment: an intrusion detection system usually prefers the false negatives of the forest over the false positives of the logistic model, while an alerting system with few analysts may prefer the opposite. All numbers here are at the default 0.5 threshold; moving the threshold trades recall against false positives and would change every figure in this section. No threshold, hyperparameter or model was selected using the test set.

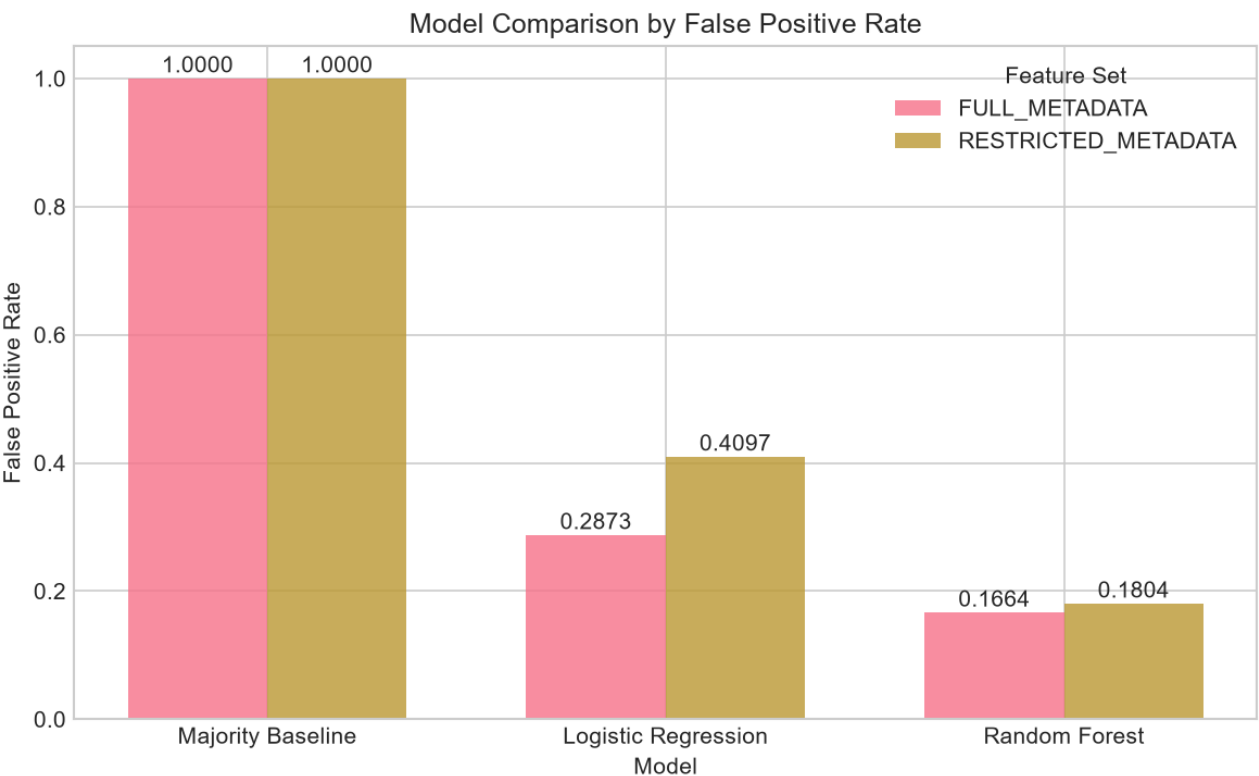
7. Figures



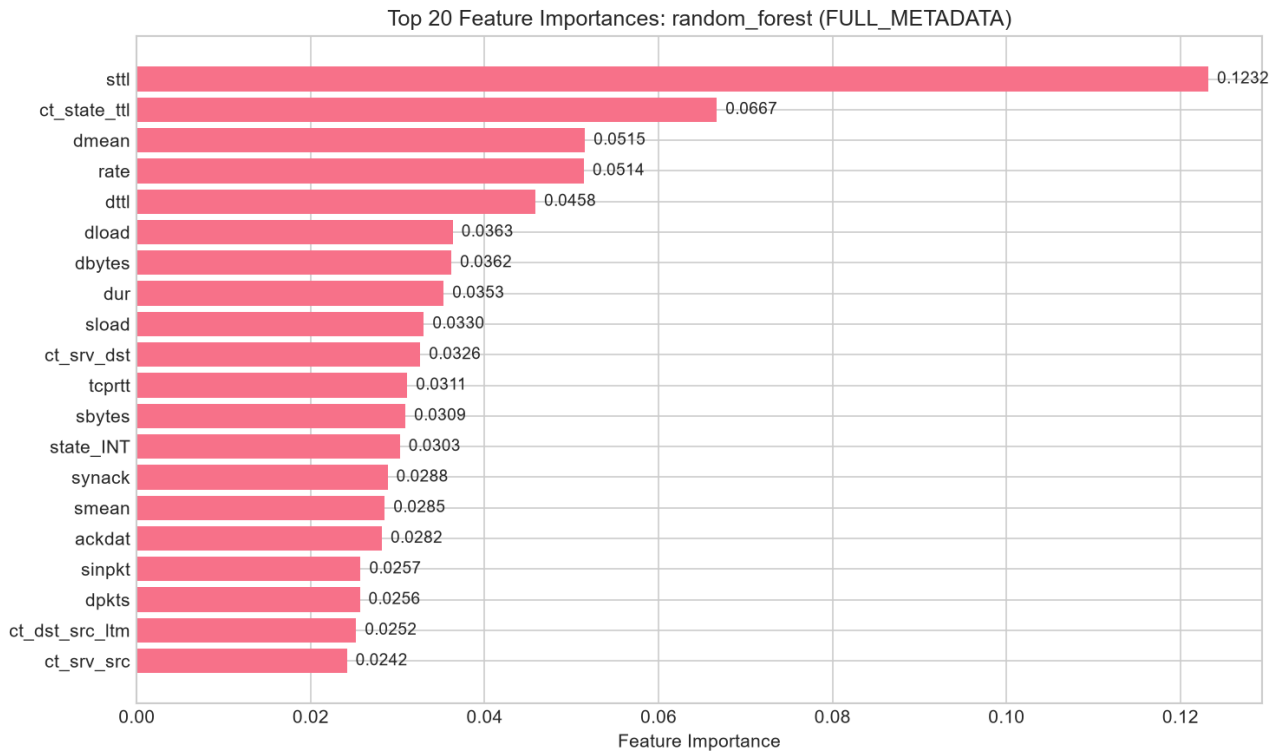
Confusion matrix - Random Forest on FULL_METADATA. Rows are true classes, columns predicted: 30,844 true negatives, 6,156 false positives, 1,398 false negatives, 43,934 true positives (test split: 37,000 normal, 45,332 anomalous).



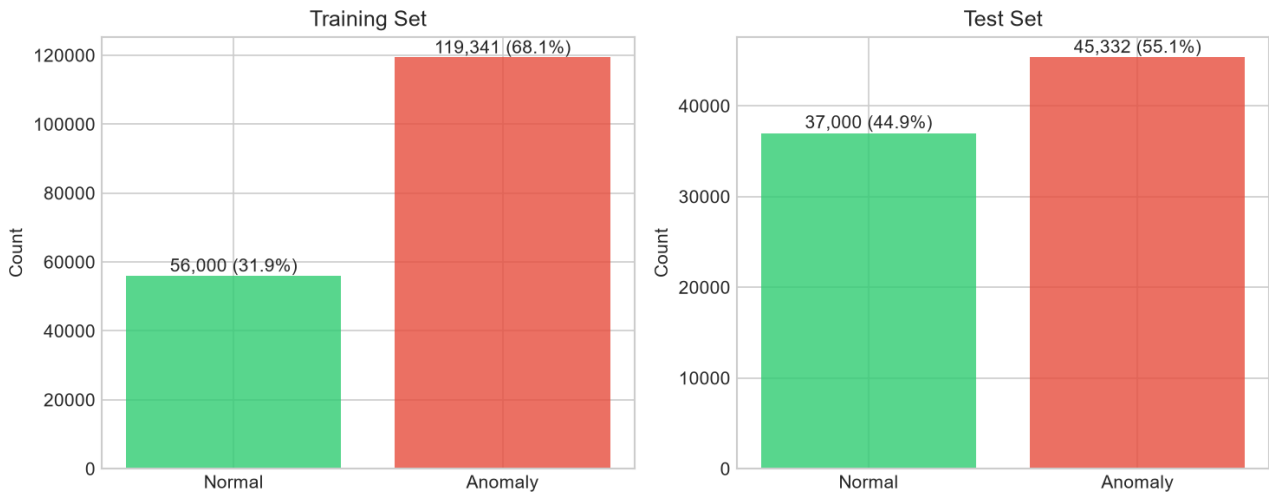
F1 by model and feature set, including the majority baseline floor.



False positive rate by model and feature set - the normal traffic each detector would mislabel as anomalous.



Feature importance of the Random Forest trained on FULL_METADATA.



Class distribution of the train and test splits actually used for this run.

8. Limitations

- Dataset age: UNSW-NB15 was captured in 2015, before TLS 1.3, QUIC and encrypted SNI became widespread; modern traffic has different metadata distributions.
- Synthetic attacks: the attack traffic was generated in a lab and does not reproduce a determined human adversary.
- Distribution shift: any real deployment sees traffic this dataset does not contain; no cross-network generalization is claimed.
- Feature availability: timing and TCP fields assume a vantage point that observes the handshake and enough packets per flow; short or sampled flows provide weaker versions of them.
- Binary simplification: nine attack categories are collapsed into one anomaly class, so per-attack behavior is not analyzed.
- Single operating point: everything is reported at threshold 0.5 with no tuning; other points on the precision-recall

curve were not explored.

- Metadata is not anonymity: traffic metadata can fingerprint and profile users even when payloads are never read. This project demonstrates payload-free detection, not a formal privacy guarantee, and makes no claim of robustness against evasion.

9. Reproduction

From a clean checkout of the repository:

- `pip install -r requirements.txt && pip install -e .`
- `python -m privacy_preserving_nad.data.download --fetch` (or place the official CSVs in `data/raw/` by hand)
- `python -m privacy_preserving_nad.pipeline` (full experiment)
- `python scripts/render_results.py` (refresh the README results block)
- `python scripts/generate_report.py` (rebuild this PDF)
- `python -m pytest tests/ -v` (test suite, runs on the development sample only - CI never downloads the full dataset)

Determinism: random seeds are fixed at None for numpy, Python's random module and every estimator, so repeating the pipeline reproduces the same metrics. Every figure and table in this report is regenerated from `results/metrics/` at build time; none of it is entered by hand.

10. Future work

- Repeat the same policy on newer captures (CIC-IDS2017, CIC-DoS2019) to test transfer beyond a 2015 laboratory dataset.
- Adversarial evaluation: measure how far an attacker can perturb metadata before detection breaks.
- Per-attack-category metrics instead of a single anomaly class.
- Threshold selection for a target false-positive budget.
- Concept drift monitoring of metadata statistics over time.
- Additional baselines (gradient boosting, simple unsupervised detectors) once these baselines are established.

11. Files this report was built from

- `configs/config.yaml` - feature sets, model parameters, seed, checksums
- `results/metrics/evaluation_results.json` - every metric and confusion matrix above
- `results/metrics/dataset_info.json` - which dataset was used, row counts, class counts
- `results/figures/*.png` - the embedded plots
- `src/privacy_preserving_nad/features/build_features.py` - feature policy text and exclusions